Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- UDP (User Datagram Protocol)
  - Connectionless transport protokolü; en basit, en hafif transport katmanı protokolü.
  - Header yapısı (8 byte):
    - Source Port (16 bit): Kaynak uygulamanın port numarası.
    - Destination Port (16 bit): Hedef uygulamanın port numarası.
    - Length (16 bit): Header + veri toplam uzunluğu (min 8 byte).
    - Checksum (16 bit): Hata tespiti; isteğe bağlı olarak 0 bırakılabilir.
  - Checksum hesabı, IP header checksum'a benzer: 1'e göre komplemanı alınarak yapılır.
  - IP'de Protocol alanında UDP = 17.
  - Fragmentation, sıralama, akış kontrolü, tıkanıklık kontrolü YOKTUR.
  - Tek parça halinde gönderilir/alınır; ideal olarak tek bir data link frame'ine sığmalıdır.
- UDP'nin Kullanım Alanları
  - **DNS**: Bir soru bir cevap, basit istek-cevap yapısı; function call benzeri.
  - **RPC (Remote Procedure Call) / RMI (Remote Method Invocation)**: Uzaktaki bir servisi çağırma; UDP üzerinde çalışabilir.
  - Gerçek zamanlı uygulamalar (ses/görüntü aktarımı) için taban olarak kullanılır (RTP gibi protokoller UDP üzerine kurulur).
  - Avantaj: Düşük overhead, hızlı kurulum, basitlik.
- RPC (Remote Procedure Call) / RMI
  - Lokaldeki bir fonksiyon çağrısını uzaktaki bir sisteme taşıma mekanizmasıdır.
  - Stub: Client ve server tarafında, parametreleri network'e uygun formata dönüştüren kod parçaları.
  - Programcı, network detaylarıyla uğraşmadan uzaktaki servisi sanki lokalde çağırıyormuş gibi yazar.
  - C'de pointer gibi yapılar sorun çıkarır (karşı tarafta anlamsız); bunun çözümü pointer'ın gösterdiği alanın tamamını göndermektir.
  - UDP ile çalışırken güvenilirlik uygulama seviyesinde sağlanmalıdır (retransmission).
  - Avantaj: abstraction, geliştirici verimliliği; dezavantaj: pointer/veri yapısı dönüşümü, performans.
- RTP (Real-time Transport Protocol)
  - UDP üzerine kurulu bir transport protokolü (paradoks gibi görünür, ama user space'te çalışan bir kütüphane).
  - Multimedia (ses, görüntü) akışı için tasarlanmıştır.
  - Temel özellikler:
    - **Sequence number**: Paket sırasını belirleme; kayıp tespiti.
    - **Timestamp**: Zaman senkronizasyonu; jitter hesaplama.
    - **Source identifier**: Hangi kaynaktan geldiğini belirleme.
    - **Payload type**: Veri tipi (ses, video, vb.).
    - **Marker**: Frame başlangıcı gibi önemli noktaları işaretleme.
  - RTCP (RTP Control Protocol): Akış kalitesi kontrolü, istatistik toplama; "RTCP'deki TCP, Transmission Control Protocol ile ilgisi yoktur" vurgusu önemlidir.
- Jitter ve Buffer Yönetimi
  - Jitter: Gecikmenin ortalama değerinden sapmalarıdır; paketler arası gecikme farkları.

  - Sabit hızda gönderilen paketler (constant rate) network'te farklı gecikmelerle ulaşır.
  - Destination'da buffer ile jitter absorbe edilir; buffer'ın sınırları vardır (sonsuz değildir).
  - Eğer buffer yetersizse, paket kaçırılır (miss) ve resync gerekir (sequence number, timestamp ile).
  - Real-time playback: Önceki ve sonraki frame arasındaki süre kadar geçerli; bu süre dışındaki buffer birikimi anlamsızdır.
- TCP'ye Giriş
  - Connection-oriented, güvenilir transport protokolü.
  - Service model: Byte stream (sıralı, hatasız, kayıpsız).
  - TCP başlıca yapılar: sequence number, ACK, sliding window, timer, retransmission.
  - Port numaraları well-known portlar (0-1023) ve dinamik portlar (1024+) olarak ikiye ayrılır.
  - FTP: 21 (kontrol) + 20 (veri); SSH: 22; SMTP: 25; DNS: 53; HTTP: 80; HTTPS: 443; POP3: 110; IMAP: 143.
  - Application-layer: TCP/IP'nin güvenilirliği, uygulamaların ihtiyacına göre tasarlanmıştır; ağ katmanının sınırları kabul edilir, üst katmanda çözüm üretilir.

Hocanın Özellikle Vurguladığı Kısımlar

- UDP'nin "işe yaramaz" olmadığı
  - UDP basit görünür ama birçok kritik uygulama için idealdir. DNS, RPC, gerçek zamanlı uygulamalar UDP kullanır.
- RPC'nin pointer sorunu
  - C gibi dillerde pointer'lar lokal adreslerdir; karşı tarafta anlamsızdır. Bu, RPC implementasyonunda dikkat edilmesi gereken önemli bir noktadır. Çözüm: pointer'ın gösterdiği alanı olduğu gibi göndermek.
- RTP'nin "transport üzerine transport" paradoksu
  - RTP, UDP üzerine kuruludur; ama aslında bir transport protokolüdür. Bu, katmanlı yapının her zaman katı olmadığını gösterir. Pratikte, RTP user space'te çalışan bir kütüphanedir.
- RTCP'nin TCP ile ilişkisi yok
  - Hoca özellikle uyarır: RTCP'deki "TCP" harfleri, Transmission Control Protocol ile ilgili değildir; bu karışıklık sorularda sıkıntı yaratabilir.
- TCP'nin temel özellikleri
  - Connection-oriented, reliable, byte stream, full-duplex.
  - Flow control (sliding window) ve congestion control mekanizmaları vardır.
  - TCP üzerinde uygulama geliştirmek için socket API kullanılır.

Kısa Tekrar Notları

- UDP: 8 byte header; port, length, checksum
- UDP: fragmentation, ordering, flow/congestion control YOK
- DNS, RPC UDP üzerinde
- RTP: UDP üzerine kurulu, user space library
- Sequence number + timestamp + source identifier = RTP temel alanları
- Jitter: gecikme varyansı; buffer ile absorbe
- TCP: byte stream, reliable, sliding window
- Well-known ports: 0-1023; HTTP=80, HTTPS=443, DNS=53, SSH=22

Detaylı Açıklamalar

UDP, transport katmanının en basit protokolüdür. Sadece 8 byte'lık bir header ekler; geri kalan her şey uygulamaya bırakılmıştır. Bu, hem avantaj hem dezavantajdır: basit olduğu için hızlıdır, ama güvenilirlik yoktur. Modern internet'te UDP, çoğu zaman "alt yapı" olarak kullanılır; üzerine RTP gibi daha akıllı protokoller kurulur.

RPC, dağıtık sistemlerin temel yapı taşlarından biridir. Bir programcı, lokalde `calculate(x, y)` çağrısı yaptığında, bu çağrı aslında uzaktaki bir sunucuya gider, orada çalışır, sonuç geri döner. Programcı network detaylarını bilmez; sadece fonksiyonu çağırır. Bu, geliştirici verimliliğini

büyük ölçüde artırır. Ancak pointer, struct, array gibi karmaşık veri tiplerinin aktarımı zorluk çıkarır; çünkü farklı sistemlerde farklı temsiller (endianness, alignment) olabilir.

RTP, gerçek zamanlı multimedia akışı için tasarlanmış bir protokoldür. UDP üzerine kurulu olması paradoks gibi görünür çünkü UDP güvenilir değildir. Ancak RTP'nin amacı, gerçek zamanlı akış için yeterli bilgiyi (sıra numarası, zaman damgası, kaynak kimliği) sağlamaktır; güvenilirlik uygulamanın sorumluluğundadır. RTP, kullanıcı alanında (user space) çalışan bir kütüphanedir, kernel'a gömülü değildir.

Jitter yönetimi, gerçek zamanlı uygulamaların en önemli sorunlarından biridir. Sabit hızda üretilen bir video akışı, network'te farklı gecikmelerle ulaşır. Eğer destination'da anında oynatılırsa, kullanıcı sık sık duraksamalar veya atlamalar görür. Çözüm: bir buffer ile jitter absorbe edilir. Ancak buffer sonsuz değildir; gerçek zamanlı akışlarda, bir frame'in geçerliliği bir sonraki frame'e kadardır. Bu nedenle, çok eski frame'ler biriktirilemez; buffer'dan zamanında oynatılmalıdır.

TCP, transport katmanının en karmaşık ve en yaygın protokolüdür. Connection-oriented yapısı, sliding window ile akış kontrolü, retransmission ile güvenilirlik sağlar. Uygulamalar, TCP üzerinden güvenli bir byte stream olarak iletişim kurar. TCP, network katmanının güvenilmezliğini tamamen soyutlar; uygulama, paket kaybı, sırasız gelme gibi sorunlarla uğraşmak zorunda kalmaz.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.